Universidade Politécnica de Viseu
Tecnologia e Gestão Viseu

eisi

Sistemas de Informação em Dispositivos Móveis

Introdução

2026/2027

APRESENTAÇÃO, AMBIENTE DE TRABALHO E A LINGUAGEM KOTLIN

A app Campo

A Campo é a aplicação que vamos desenvolver nas aulas. Destina-se a equipas de técnicos que registam ocorrências no terreno, cada uma com título, descrição, estado e localização. Como no terreno a rede falha com frequência, a app tem de funcionar sem ligação, sincronizar quando a ligação regressa e lidar com o caso em que dois técnicos alteram a mesma ocorrência.

- O backend usado é o Supabase, que nos dá uma base de dados Postgres, autenticação e políticas de acesso linha a linha.
- Vamos construí-la primeiro em Kotlin com Jetpack Compose e depois em Swift com SwiftUI.
- No final, passamos a lógica comum para um módulo Kotlin Multiplatform.

Ambiente de trabalho

Para as aulas de Android precisamos de três coisas a funcionar desde já:

1. O Android Studio, na versão estável mais recente, com um emulador criado (ex. Pixel 9).
2. Uma conta gratuita no Supabase. Cada aluno cria aí um projeto para o seu trabalho.
    1. O plano gratuito só permite 2 projetos ativos, por isso, considere reservar um projeto para o trabalho de grupo.
3. Uma conta no GitHub e um repositório.

Dois comandos permitem confirmar rapidamente que a instalação está em ordem:

```
adb devices                 # o emulador deve aparecer na lista
./gradlew --version         # Gradle 9.x, com JDK 17 ou superior
```

Segredos fora do repositório

O endereço do projeto Supabase e a respetiva *publishable key* guardam-se num ficheiro que o git ignora. No Android esse ficheiro é o `local.properties`, de onde o Gradle copia os valores para a classe `BuildConfig`. No iOS o papel equivalente cabe ao `Secrets.xcconfig`.

```
# local.properties (nunca entra no git)
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_PUBLISHABLE_KEY=sb_publishable_...
```

Convém ter presente que é possível extrair a *publishable key* de um APK. Não é ela que protege os dados, e por isso a segurança vai assentar nas políticas definidas no servidor, que estudamos na aula 4. Já a *secret key* (`sb_secret_…`, a antiga `service_role`) dá acesso total à base de dados e nunca pode ser incluída numa app.

O que é o Kotlin

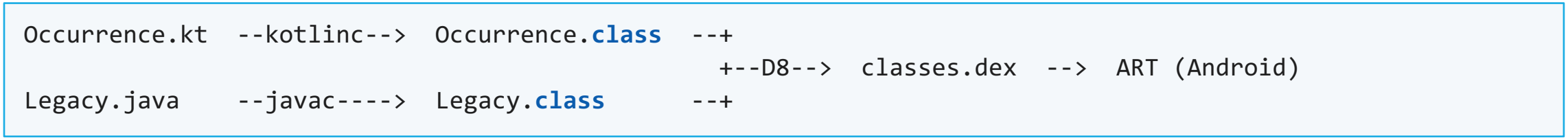
O Kotlin é uma linguagem de tipagem estática criada pela JetBrains, a empresa que desenvolve o IntelliJ IDEA, sobre o qual o Android Studio é construído. Foi apresentada em 2011 e chegou à versão 1.0 em 2016. Em 2017 a Google passou a suportá-la oficialmente no Android e, desde 2019, a documentação, os exemplos e as bibliotecas novas da plataforma são escritos primeiro em Kotlin.

Nasceu da experiência de quem escrevia muito Java e queria uma linguagem mais concisa e mais segura sem abandonar o ecossistema existente. Daí resultam as suas características mais marcantes:

- interoperabilidade total com Java, pelo que qualquer biblioteca Java pode ser usada a partir de Kotlin e vice-versa;
- distinção, no sistema de tipos, entre valores que podem e que não podem ser nulos;
- menos código repetitivo: inferência de tipos, classes de dados e funções como valores;
- suporte na própria linguagem para programação assíncrona, com as coroutines.

Kotlin - como e onde?

Tal como o Java, o Kotlin compila para bytecode da JVM. No Android, esse bytecode é depois convertido para o formato DEX, que a máquina virtual do sistema (ART) executa. É por partilharem o mesmo bytecode que as duas linguagens podem coexistir no mesmo projeto.



O compilador tem também outros destinos. O Kotlin/Native gera código máquina, por exemplo para iOS, e existem ainda destinos para JavaScript e WebAssembly. É esta capacidade que está na base do Kotlin Multiplatform.

Para experimentar a linguagem sem criar um projeto, podem usar o editor em play.kotlinlang.org.

Um primeiro programa

Antes de olharmos para os detalhes, vale a pena ver um programa completo. Repare-se que a função `main` não está dentro de nenhuma classe, que não há ponto e vírgula no fim das linhas e que o tipo vem depois do nome, separado por dois pontos.

```
fun main() {
    val team = "Norte"
    val open = countOpen(listOf("aberta", "fechada", "aberta"))
    println("Equipa $team: $open ocorrências abertas")
}

fun countOpen(states: List<String>): Int {
    var total = 0
    for (state in states) {
        if (state == "aberta") total++
    }
    return total
}
```

Quem conhece Java reconhece quase tudo. Nos slides seguintes percorremos o que é diferente.

Valores, variáveis e tipos

Em Kotlin, a forma habitual de declarar um nome é com `val`, que não permite voltar a atribuir. Reserva-se o `var` para os casos em que o valor tem mesmo de mudar. O tipo raramente precisa de ser escrito, porque o compilador o deduz a partir do valor atribuído.

```
val title = "Poste partido"          // String, não muda
var attempts = 0                     // Int, vai mudar
attempts += 1

val teamId: String = "t-norte"       // tipo explícito, quando ajuda a leitura
val summary = "$title, tentativa $attempts"
val size = "${title.length} caracteres"
```

Dentro de uma string, `$nome` insere o valor de uma variável e `${expressão}` o resultado de uma expressão. Deixa de ser necessário concatenar com `+`.

`if` e `when` como expressões

Em Kotlin, o `if` e o `when` produzem um valor e podem por isso ser usados do lado direito de uma atribuição. O `when` ocupa o lugar do `switch` do Java, com a vantagem de aceitar condições, intervalos e tipos e de não precisar de `break`.

```
val label = if (attempts == 0) "primeira vez" else "repetição"

fun describe(count: Int): String = when {
    count == 0 -> "sem ocorrências"
    count in 1..9 -> "poucas"
    else -> "muitas"
}

fun statusLabel(status: Status): String = when (status) {
    Status.OPEN -> "Aberta"
    Status.IN_PROGRESS -> "Em curso"
    Status.CLOSED -> "Fechada"
}
```

Null safety

O tipo `String` nunca admite o valor nulo, ao contrário do tipo `String?`. Quando um valor pode ser nulo, o compilador obriga-nos a tratar esse caso antes de o usarmos. A linguagem oferece quatro operadores para o fazer.

```
val photo: String? = occurrence.photoPath

val length = photo?.length              // Int?: fica nulo se photo for nulo
val name = photo ?: "sem foto"          // valor a usar quando é nulo
photo?.let { upload(it) }               // o bloco só corre se não for nulo
val forced = photo!!.length             // lança NullPointerException se for nulo
```

O operador `!!` desliga a verificação do compilador e deve ser evitado.

Funções, parâmetros nomeados e valores por omissão

Quando o corpo de uma função é uma única expressão, pode escrever-se a seguir a um `=`, sem chavetas nem `return`. Os parâmetros podem ter um valor por omissão e ser indicados pelo nome na chamada, o que evita as várias versões sobrecarregadas do mesmo método que é habitual escrever em Java.

```
fun formatTitle(title: String, status: Status, upper: Boolean = false): String {
    val text = "${title.trim()} · ${statusLabel(status)}"
    return if (upper) text.uppercase() else text
}

fun greet(name: String) = "Olá, $name"

formatTitle("Poste partido", Status.OPEN)
formatTitle(title = "Fuga de água", status = Status.CLOSED, upper = true)
```

data class

Uma `data class` é uma classe cuja finalidade é guardar dados. A partir das propriedades declaradas no construtor, o compilador gera os métodos `equals`, `hashCode`, `toString` e `copy`, que em Java teríamos de escrever ou gerar com o IDE.

```
data class Occurrence(
    val id: String,
    val title: String,
    val status: Status = Status.OPEN,
    val version: Int = 1,
)

val o1 = Occurrence("o1", "Poste partido")
val o2 = o1.copy(status = Status.CLOSED)      // novo objeto; o1 fica igual
println(o1 == o2)                             // false: compara o conteúdo
println(o2)                                   // Occurrence(id=o1, title=Poste partido, ...)
```

Como as propriedades são `val`, o objeto é imutável. Para o "alterar", cria-se uma cópia com a diferença pretendida, usando `copy`.

`enum class` e `sealed interface`

Um `enum` representa um conjunto fixo de valores. Quando cada caso precisa de transportar dados diferentes, usa-se uma `sealed interface`. Como o compilador conhece todas as implementações, verifica se o `when` trata cada uma delas.

```
enum class Status { OPEN, IN_PROGRESS, CLOSED }

sealed interface LoadResult {
    data object Loading : LoadResult
    data class Success(val items: List<Occurrence>) : LoadResult
    data class Failure(val message: String) : LoadResult
}

fun render(result: LoadResult): String = when (result) {
    LoadResult.Loading -> "a carregar"
    is LoadResult.Success -> "${result.items.size} ocorrências"
    is LoadResult.Failure -> result.message      // sem else: se faltar um caso, não compila
}
```

Lambdas e funções de ordem superior

Uma lambda é uma função sem nome, escrita entre chavetas, que pode ser guardada numa variável ou passada como argumento. Às funções que recebem outras funções chama-se funções de ordem superior. Se a lambda for o último argumento, escreve-se fora dos parênteses.

```
val isOpen: (Occurrence) -> Boolean = { it.status == Status.OPEN }

fun <T> List<T>.countWhere(predicate: (T) -> Boolean): Int {
    var n = 0
    for (item in this) if (predicate(item)) n++
    return n
}

occurrences.countWhere { it.status == Status.OPEN }   // lambda fora dos parênteses
occurrences.countWhere(isOpen)                        // ou uma referência guardada
```

Quando a lambda tem um só parâmetro, este pode ser referido por `it`. Com dois ou mais, é preciso dar-lhes nome: `{ a, b -> a + b }`.

Coleções

As interfaces `List`, `Set` e `Map` só permitem leitura. Para obter uma coleção que possa ser alterada usam-se funções como `mutableListOf`. As operações mais comuns podem ser encadeadas e devolvem sempre uma coleção nova, deixando a original intacta.

```
val open = occurrences.filter { it.status == Status.OPEN }
val titles = occurrences.map { it.title }
val recent = occurrences.sortedByDescending { it.updatedAt }
val byStatus = occurrences.groupBy { it.status }        // Map<Status, List<Occurrence>>
val first = occurrences.first { it.teamId == "t-norte" }
val maybe = occurrences.firstOrNull { it.id == "x" }    // null em vez de exceção

occurrences
    .filter { it.status != Status.CLOSED }
    .sortedBy { it.title }
    .map { it.title }
```

`object`, `companion` e extension functions

A palavra `object` declara uma classe com uma única instância. Dentro de uma classe, o `companion object` reúne aquilo que em Java seria `static`. Já uma extension function permite acrescentar um método a um tipo que não nos pertence, como `String` ou `List`, sem recorrer a herança.

```
object Ids { fun next(): String = UUID.randomUUID().toString() }

data class Occurrence(val title: String, val status: Status) {
    companion object { const val TITLE_MIN_LENGTH = 3 }
}

fun String.isValidTitle(): Boolean = trim().length >= Occurrence.TITLE_MIN_LENGTH
fun List<Occurrence>.openOnly(): List<Occurrence> = filter { it.status == Status.OPEN }

"ab".isValidTitle()        // false
sampleOccurrences.openOnly().size
```

Coroutines: `suspend`, `launch` e `delay`

Uma função marcada com `suspend` pode ficar à espera de uma operação demorada, como um pedido de rede, sem bloquear a thread em que corre. Só pode ser chamada a partir de outra função `suspend` ou de dentro de uma coroutine, que se inicia com `launch` num determinado escopo.

```
suspend fun fetchOccurrences(): List<Occurrence> {
    delay(500)                    // simula a rede; a thread fica livre entretanto
    return sampleOccurrences
}

fun main() = runBlocking {
    launch {
        val items = fetchOccurrences()
        println("${items.size} ocorrências")
    }
    println("isto é impresso primeiro")
}
```

No Android não precisamos de criar o escopo: os ViewModels dispõem de `viewModelScope` e as Activities de `lifecycleScope`.

Flow

Um `Flow` representa uma sequência de valores que vão chegando ao longo do tempo. Só começa a produzir valores quando alguém o coleta.

```
fun ticker(): Flow<Int> = flow {
    repeat(3) { n ->
        delay(200)
        emit(n)
    }
}

suspend fun demo() {
    ticker().collect { n -> println("tick $n") }
}
```

Kotlin para quem conhece Java

Java	Kotlin
`final String s = "x";`	`val s = "x"`
`String s = null;`	`val s: String? = null`
`if (s != null) s.length()`	`s?.length`
`s != null ? s : "x"`	`s ?: "x"`
`switch` com `break`	`when`, usado como expressão
classe com getters, `equals` e `hashCode`	`data class`
`static`	`companion object` ou função fora de uma classe
`List<String> l = new ArrayList<>()`	`val l = mutableListOf<String>()`
`Thread`, `AsyncTask`, callbacks	funções `suspend` e coroutines
ponto e vírgula no fim da linha	não se escreve

Anatomia de um projeto Android Studio

O assistente *Empty Activity* cria um projeto Gradle com um módulo chamado app. As versões das bibliotecas ficam centralizadas num catálogo, e a Activity limita-se a entregar o ecrã a uma função Compose.

```
campo-android/
├── settings.gradle.kts          módulos e repositórios
├── build.gradle.kts             plugins usados no projeto
├── gradle/libs.versions.toml    catálogo de versões e bibliotecas
└── app/
    ├── build.gradle.kts         applicationId, minSdk 26, dependências
    └── src/main/java/com/estgv/campo/MainActivity.kt
```

```kotlin
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { CampoTheme { Text("Campo") } }
    }
}
```

Exercícios

E1.1 - Contagem por estado. Escrever `fun List<Occurrence>.countByStatus(): Map<Status, Int>`, recorrendo a `groupBy` e `mapValues`. Está concluído quando um teste com as três ocorrências de exemplo obtém `{OPEN=1, IN_PROGRESS=1, CLOSED=1}`.

E1.2 - Coordenadas opcionais. Escrever `fun Occurrence.coordinates(): String?`, que devolve `"40.66, -7.91"` apenas quando `lat` e `lng` existem. Deve usar `?.let` e não pode usar `!!`. Está concluído quando passam dois testes, um com coordenadas e outro com `lat = null`.

E1.3 · Validação com `sealed interface`. Definir `ValidationResult` com os casos `Valid` e `Invalid(reason)` e a função `validateTitle(title: String): ValidationResult`. Está concluído quando `"ab"` dá `Invalid`, `"  Poste  "` dá `Valid` e o `when` sobre o resultado não precisa de `else`.